Unicode in the Library, Part 1: UTF Transcoding

Document #:	P2728R8 [Latest] [Status]
Date:	2025-10-06
Project:	Programming Language C++
Audience:	SG-16 Unicode SG-9 Ranges LEWG

Reply-to:	Eddie Nolan <eddiejnolan@gmail.com>

Contents

§ 1 High-Level Overview

This paper introduces views and ranges for transcoding between UTF formats:

```
static_assert((u8"🙂" | views::to_utf32 | ranges::to<u32string>()) == U"🙂");
```

It handles errors by replacing invalid subsequences with `U"\uFFFD"`:

```
static_assert((u8"🙂" | views::take(3) | to_utf32 | ranges::to<std::u32string>()) ==
U"\uFFFD");
```

And by providing `or_error` views that provide `std::expected`:

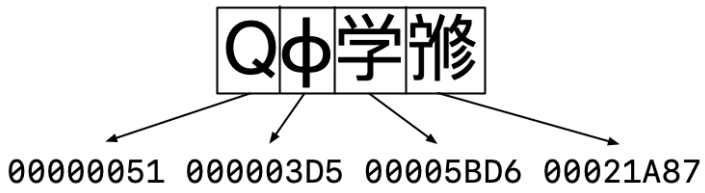
```
static_assert(
  *(u8"🙂" | views::take(3) | views::to_utf32_or_error).begin() ==
  unexpected{utf_transcoding_error::truncated_utf8_sequence});
```

§ 2 UTF Primer

If you're already familiar with Unicode, you can skip this section.

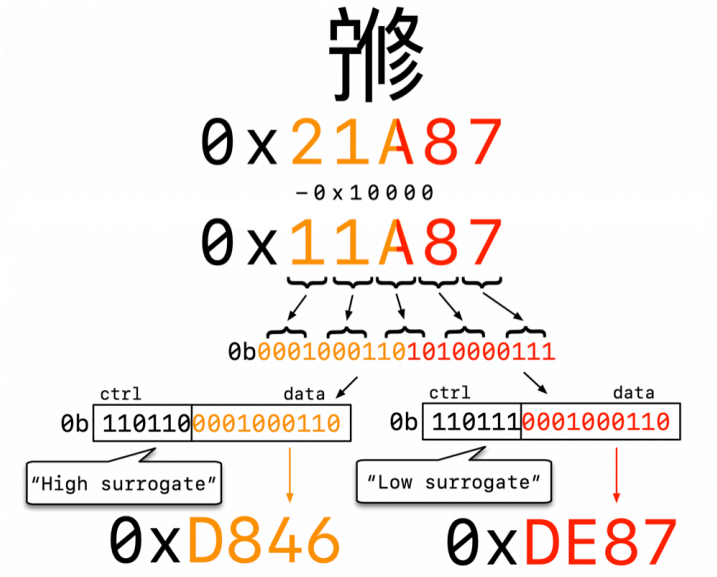
The Unicode standard maps *abstract characters* to *code points* in the *Unicode codespace* from `0` to `0x10FFFF`. Unicode text forms a *coded character sequence*, "an ordered sequence of one or more code points." [Definitions]

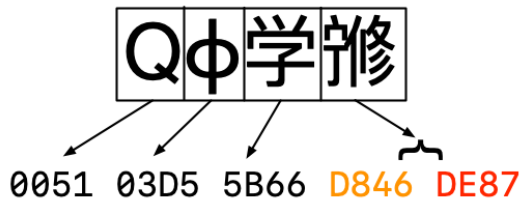
The simplest way of encoding code points is UTF-32, which encodes code points as a sequence of 32-bit unsigned integers. The building blocks of an encoding are *code units*, and UTF-32 has the most direct mapping between code points and code units.



Any values greater than `0x10FFFF` are rejected by validators for being outside the range of valid Unicode.

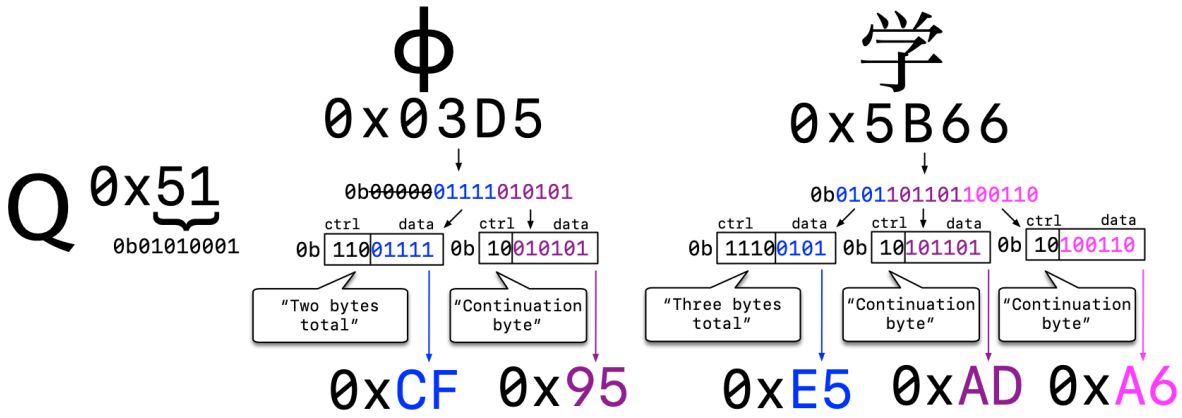
Next is UTF-16, which exists for the historical reason that the Unicode codespace used to top out at `0xFFFF`. Code points outside this range are represented using *surrogates*, a reserved area in codespace which allows combining the low 10 bits of two code units to form a single code point.

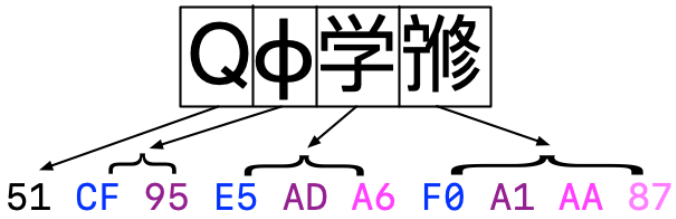




UTF-16 is rendered invalid by improper use of surrogates: a high surrogate not followed by a low surrogate or a low surrogate not preceded by a high surrogate. Note that the presence of any surrogate code points in UTF-32 is also invalid.

Finally, UTF-8, the most ubiquitous and most complex encoding. This uses 8-bit code units. If the high bit of the code unit is unset, the code unit represents its ASCII equivalent for backwards compatibility. Otherwise the code unit is either a start byte, which describes how long the subsequence is (two to four bytes long), or a continuation byte, which fills out the subsequence with the remaining data.





UTF-8 code unit sequences can be invalid for many reasons, such as a start byte not followed by the correct number of continuation bytes, or a UTF-8 subsequence that encodes a surrogate.

Transcoding in this context refers to the conversion of characters between these three encodings.

§ 3 Existing Standard UTF Interfaces in C and C++

§ 3.1 C

C contains an alphabet soup of transcoding functions in <stdlib.h>, <wchar.h>, and <uchar.h>. [Null-terminated multibyte strings]

This paper doesn't fully litigate these functions' flaws (see WG14 [N2902] for a more detailed explanation). Some of the issues users encounter include reliance on an internal global conversion state, reliance on the current setting of the global C locale, optimization barriers in one-code-unit-at-a-time function calls, and inadequate error handling that does not support replacement of invalid subsequences with as specified by Unicode.

Example:

```
setlocale(LC_ALL, "en_US.utf8");
char c[5] = {0};
const char16_t* w = u"\xd83d\xdd74";
mbstate_t state;
memset(&state, 0, sizeof(state));
c16rtomb(c, w[0], &state);
c16rtomb(c, w[1], &state);
const char* e = "\xf0\x9f\x95\xb4";
assert(strcmp(c, e) == 0);
```

§ 3.2 C++

C++'s existing transcoding functionality, other than the aforementioned functions it inherits from C, consists of the set of `std::codecvt` facets provided in `<locale>` and `<codecvt>`.

Example:

```
std::wstring_convert<std::codecvt_utf8<char32_t>, char32_t> conv;
std::string c = conv.to_bytes(U"🙂");
assert(c == "\xf0\x9f\x99\x82");
```

All of the Unicode-specific functionality in this header was deprecated in C++17, and [P2871R3] and [P2873R2] finally remove most of it in C++26. There are many concerns about these interfaces, particularly with respect to safety.

These functions throw exceptions on encountering invalid UTF. Unicode functions that use exceptions for error handling are a well-known footgun because users consistently invoke them on untrusted user input without handling the exceptions properly, leading to denial-of-service vulnerabilities.

An example of this anti-pattern (although not involving these specific functions) can be found in [CVE-2007-3917], where a multiplayer RPG server could be crashed by malicious users sending invalid UTF. Below is the patch: [wesnoth]

```
- msg = font::word_wrap_text(msg,font::SIZE_SMALL,map_outside_area().w*3/4);
+ try {
+     // We've had a joker who send an invalid utf-8 message to crash clients
+     // so now catch the exception and ignore the message.
+     msg = font::word_wrap_text(msg,font::SIZE_SMALL,map_outside_area().w*3/4);
+ } catch (utils::invalid_utf8_exception&) {
+     LOG_STREAM(err, engine) << "Invalid utf-8 found, chat message is ignored.\n";
+     return;
+ }
```

Because it doesn't use exceptions, the functionality proposed by this paper can serve as a safe, modern replacement for the deprecated and removed `codecvt` facets.

§ 4 Replacing Ill-Formed Subsequences with "🙂"

When a transcoder encounters an invalid subsequence, the modern best practice is to replace it in the output with one or more 🙂 characters (`U+FFFD REPLACEMENT CHARACTER`). The methodology for doing so is described in §3.9.6 of the Unicode Standard v17.0, Substitution of Maximal Subparts [Substitution].

For UTF-32 and UTF-16, each invalid code unit is replaced by an individual 🙂 character.

For UTF-8, the same rule applies except if "a sequence of two or three bytes is a truncated version of a sequence which is otherwise well-formed to that point." In the latter case, the full two-to-three byte subsequence is replaced by a single 🙂 character.

For example, UTF-8 encodes 🙂 as `0xF0 0x9F 0x99 0x82`.

If that sequence of bytes is truncated to just `0xF0 0x9F 0x99`, it becomes a single 🙂 replacement character.

On the other hand, if the first byte of the four-byte sequence is changed from `0xF0` to `0xFF`, then it's replaced by four replacement characters, ����, because no valid UTF-8 subsequence begins with `0xFF`.

More subtly, the subsequence `0xED 0xA0` must be replaced with two replacement characters, ��, because any continuation of that subsequence can only result in a surrogate code point, so it can't prefix any valid subsequence.

Each of the proposed `to_utfN_view` views adheres to this specification. The `to_utfN_as_error` views also use this scheme but produce `unexpected<utf_transcoding_error>` values instead of replacement characters.

§ 5 Design Overview

§ 5.1 Transcoding Views

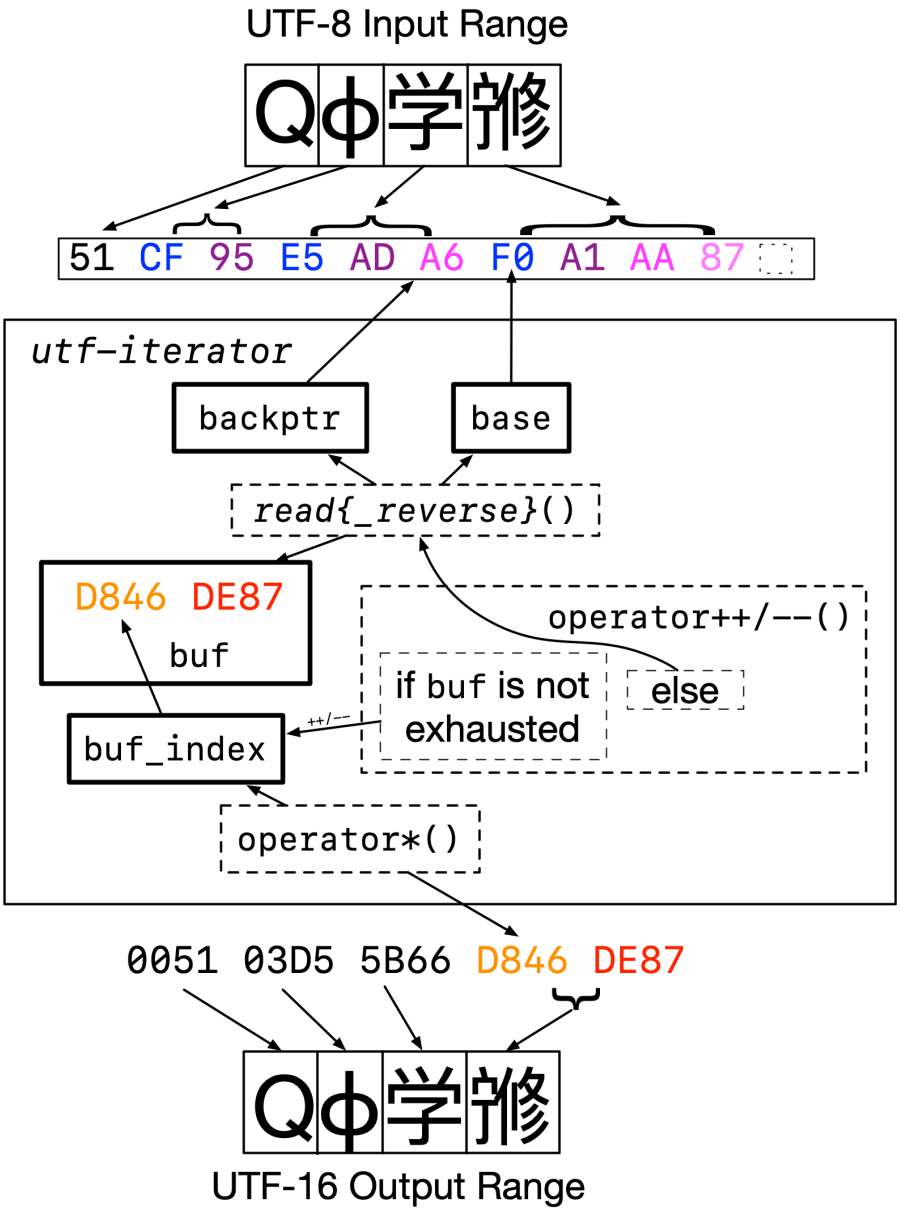
Invoking `begin()` or `end()` on a transcoding view constructs an instance of an exposition-only *`utf-iterator`* type.

The *`utf-iterator`* stores an iterator pointing to the start of the character it's transcoding, and a back-pointer to the underlying range in order to bounds check its beginning and end (which is required for correctness, not just safety).

The *`utf-iterator`* maintains a small buffer (`buf`) containing between one and four code units, which comprise the current character in the target encoding.

It also maintains an index (`buf_index`) into this buffer, which it increments or decrements when `operator++` or `operator--` is invoked, respectively. If it runs out of code units in the buffer, it reads more elements from the underlying view. `operator*` provides the current element of the buffer.

Below is an approximate block diagram of the iterator. Bold lines denote actual data members of the iterator; dashed lines are just function calls.



The *utf-iterator* is converting the string `Qф学𩸽` from UTF-8 to UTF-16. The user has iterated the view to the first UTF-16 code unit of the fourth character. `base` points to the start of the fourth character in the input. `buf` contains both UTF-16 code units of the fourth character; `buf_index` keeps track of the fact that we're currently pointing to the first one. If we invoke `operator++` on the *utf-iterator*, it will increment `buf_index` to point to the second code unit. On the other hand, if we invoke `operator--`, it will notice that `buf_index` is already at the beginning and move backward from the fourth character to the third character by invoking *read_reverse()*. The *read()* and *read_reverse()* functions contain most of the actual transcoding logic, updating `base` and filling `buf` up with the transcoded characters.

Iterating a bidirectional transcoding view backwards produces, in reverse order, the exact same sequence of characters or `expected` values as are produced by iterating the view forwards.

§ 5.1.1 `utf_transcoding_error`

Each transcoding view, like `to_utf8_view`, which produces a range of `char8_t` and handles errors by substituting `�` replacement characters, has a corresponding `_or_error` equivalent, like `to_utf8_view_or_error`, which produces a range of `expected<char8_t, utf_transcoding_error>` and handles errors by substituting `unexpected<utf_transcoding_error>`s.

`utf_transcoding_error` is an enumeration whose enumerators are:

- `truncated_utf8_sequence`
  - An ill-formed subsequence that matches the beginning of some well-formed sequence.
  - Example invalid code unit sequence: UTF-8 `0xE1 0x80`.
- `unpaired_high_surrogate`
  - Example invalid code unit sequence: UTF-16 `0xD800`.
- `unpaired_low_surrogate`
  - Example invalid code unit sequence: UTF-16 `0xDC00`.
- `unexpected_utf8_continuation_byte`
  - Example invalid code unit sequence: UTF-8 `0x80`.
- `overlong`
  - An overlong UTF-8 encoding.
  - Example invalid code unit sequence: UTF-8 `0xE0 0x80`.
- `encoded_surrogate`
  - Applies to both UTF-8 and UTF-32.
  - Example invalid code unit sequence: UTF-8 `0xED 0xA0`, UTF-32 `0x0000D800`.
- `out_of_range`
  - Applies to both UTF-8 and UTF-32
  - In UTF-8, this applies to `0xF4` if it is followed by a continuation byte greater than `0x8F`
  - In UTF-32, this is any code unit greater than `0x10FFFF`
  - Example invalid code unit sequence: UTF-8 `0xF4 0x90`, UTF-32 `0x110000`.
- `invalid_utf8_leading_byte`
  - In UTF-8, this applies to `0xC0-0xC1` and `0xF5-0xFF`.
  - Example invalid code unit sequence: UTF-8 `0xC0`.

An alternative approach to minimize the number of enumerators could merge `truncated_utf8_sequence` with `unpaired_high_surrogate` and merge `unexpected_utf8_continuation_byte` with `unpaired_low_surrogate`, but based on feedback, splitting these up seems to be preferred.

The table below compares the error handling behavior of the `to_utf16` and `to_utf16_or_error` views on various sample UTF-8 inputs from the "Substitution of Maximal Subparts" section of the Unicode standard: [SubstitutionExamples]

Table 3-8. U+FFFD for Non-Shortest Form Sequences

UTF-8 input	C0	AF	E0	80	BF	F0	81	82	41
to_utf16	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	0041
to_utf16_or_error	invalid_utf8_leading_byte	unexpected_utf8_continuation_byte	overlong	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	overlong	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	0041

Table 3-9. U+FFFD for Ill-Formed Sequences for Surrogates

UTF-8 input	E0	A0	80	ED	BF	BF	ED	AF	41
to_utf16	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	0041
to_utf16_or_error	encoded_surrogate	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	encoded_surrogate	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	encoded_surrogate	unexpected_utf8_continuation_byte	0041

Table 3-10. U+FFFD for Other Ill-Formed Sequences

UTF-8 input	F4	91	92	93	FF	41	80	BF	42
to_utf16	FFFD	FFFD	FFFD	FFFD	FFFD	0041	FFFD	FFFD	0042
to_utf16_or_error	out_of_range	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	invalid_utf8_leading_byte	0041	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	0042

Table 3-11. U+FFFD for Truncated Sequences

UTF-8 input	E1	80	E2	F0	91	92	F1	BF	41
to_utf16	FFFD		FFFD	FFFD			FFFD		0041
to_utf16_or_error	truncated_utf8_sequence		truncated_utf8_sequence	truncated_utf8_sequence			truncated_utf8_sequence		0041

§ 5.1.2 Why there are three `to_utfN_views` and no `to_utf_view`

The views in `std::ranges` are constrained to accept only `std::ranges::view` template parameters. However, they accept `std::ranges::viewable_ranges` in practice, because they each have a deduction guide that looks like this:

```
template<class R>
to_utf8_view(R &&) -> to_utf8_view<views::all_t<R>>;
```

An alternative design is possible where the `to_utfN_views` are defined in terms of a `to_utf_view` with a format NTTP, as was done in a previous version of this paper:

```
template<format Format, class R>
to_utf_view(R &&) -> to_utf_view<Format, views::all_t<R>>;

template<class V>
using to_utf8_view = to_utf_view<format::utf8, V>;
template<class V>
using to_utf16_view = to_utf_view<format::utf16, V>;
template<class V>
using to_utf32_view = to_utf_view<format::utf32, V>;
```

Although [P1814R0] would make these guides work perfectly well for `to_utf8_view` and its siblings, it's not actually possible to make use of the deduction guide for `to_utf_view` without going through one of those aliases. Having a view with this property in the standard library would break with precedent; the version of the "`to_utf_view`" concept in this paper is an exposition-only implementation detail for that reason.

However, this issue doesn't apply to the CPOs, so users are still free to write `generic_string | to_utf<char8_t>`.

§ 5.2 Code Unit Views

SG16 has a goal to ensure that C++ standard library functions that expect UTF-encoded input do not accept parameters of type `char` or `wchar_t`, whose encodings are implementation-defined, and instead use `char8_t`, `char16_t`, and `char32_t`. These views follow that pattern.

Because virtually all UTF-8 text processed by C++ is stored in `char` (and similarly for UTF-16 and `wchar_t`), this means that we need a terse way to smooth over the transition for users. To do so, this paper introduces views for casting to the `charN_t` types: `as_char8_t`, `as_char16_t`, and `as_char32_t`.

These are syntactic sugar for producing a `std::ranges::transform_view` with an exposition-only transformation functor that performs the needed cast.

§ 6 Additional Examples

§ 6.1 Transcoding a UTF-8 string literal to a `std::u32string`

```
std::u32string hello_world =
  u8"こんにちは世界" | std::views::to_utf32 | std::ranges::to<std::u32string>();
```

§ 6.2 Sanitizing potentially invalid Unicode

Note that transcoding to and from the same encoding is not a no-op; it must maintain the invariant that the output of a transcoding view is always valid UTF.

```
template <typename CharT>
std::basic_string<CharT> sanitize(CharT const* str) {
   return std::null_term(str) | std::views::to_utf<CharT> | std::ranges::to<std::ba-
        sic_string<CharT>>();
}
```

§ 6.3 Returning the final non-ASCII code point in a string, transcoding backwards lazily:

```
std::optional<char32_t> last_nonascii(std::ranges::view auto str) {
  for (auto c : str | std::views::to_utf32 | std::views::reverse
                    | std::views::filter([](char32_t c) { return c > 0x7f; })) {
    return c;
  }
  return std::nullopt;
}
```

§ 6.4 Transcoding strings and throwing a descriptive exception on invalid UTF

(This assumes a reflection-based `enum_to_string` function.)

```
template <typename FromChar, typename ToChar>
std::basic_string<ToChar>  transcode_or_throw(std::basic_string_view<FromChar>  input)
        {
  std::basic_string<ToChar> result;
  auto view = input | std::views::to_utf_or_error<ToChar>;
  for (auto it = view.begin(), end = view.end(); it != end; ++it) {
    if ((*it).has_value()) {
      result.push_back(**it);
```

```
    } else {
      throw std::runtime_error("error at position " +
                               std::to_string(it.base() - input.begin()) + ": " +
                               enum_to_string((*it).error()));
    }
  }
  return result;
}
```

```
  // prints: "error at position 2: truncated_utf8_sequence"
  transcode_or_throw<char8_t, char16_t>(
    u8"hi🙂" | std::views::take(5) | std::ranges::to<std::u8string>());
```

§ 6.5 Changing the suits of Unicode playing card characters:

```
enum class suit : std::uint8_t {
  spades = 0xA,
  hearts = 0xB,
  diamonds = 0xC,
  clubs = 0xD
};

// Unicode playing card characters are laid out such that changing the second least
// significant nibble changes the suit, e.g.
// U+1F0A1 PLAYING CARD ACE OF SPADES
// U+1F0B1 PLAYING CARD ACE OF HEARTS
constexpr char32_t change_playing_card_suit(char32_t card, suit s) {
  if (U'\N{PLAYING CARD ACE OF SPADES}' <= card && card <= U'\N{PLAYING CARD KING OF
        CLUBS}') {
    return (card & ~(0xF << 4)) | (static_cast<std::uint8_t>(s) << 4);
  }
  return card;
}

void change_playing_card_suits() {
  std::u8string_view const spades = u8"🂡🂢🂣🂤🂥🂦🂧🂨🂩🂪🂫🂭🂮";
  std::u8string const hearts =
    spades |
    to_utf32 |
    std::views::transform(std::bind_back(change_playing_card_suit, suit::hearts)) |
    to_utf8 |
    std::ranges::to<std::u8string>();
  assert(hearts == u8"🂱🂲🂳🂴🂵🂶🂷🂸🂹🂺🂻🂽🂾");
}
```

§ 7 Dependencies

The code unit views depend on [P3117R1] “Extending Conditionally Borrowed”.

§ 8 Implementation Experience

The most recent revision of this paper has a reference implementation called beman.utf_view available on GitHub, which is a fork of Jonathan Wakely’s implementation of P2728R6 as an implementation detail for libstdc++. It is part of the Beman project.

Versions of the interfaces provided by previous revisions of this paper have also been implemented, and re-implemented, several times over the last 5 years or so, as part of a proposed (but not yet accepted!) Boost library, Boost.Text. Boost.Text has hundreds of stars on GitHub.

Both libraries have comprehensive tests.

§ 9 Details and Pseudo-wording

§ 9.1 Exposition-only concepts and traits

```
namespace std::ranges {

  template<class T>
  constexpr bool is-empty-view = false;
  template<class T>
  constexpr bool is-empty-view<ranges::empty_view<T>> = true;

  template<class T>
  concept code-unit =
        same_as<remove_cv_t<T>,  char8_t>  ||  same_as<remove_cv_t<T>,  char16_t>  ||
        same_as<remove_cv_t<T>, char32_t>;

  template<class T>
  concept utf-range = ranges::input_range<T> && code-unit<ranges::range_value_t<T>>;

  template<class I>
  consteval auto bidirectional-at-most() { // exposition only
    if constexpr (bidirectional_iterator<I>) {
      return bidirectional_iterator_tag{};
    } else if constexpr (forward_iterator<I>) {
      return forward_iterator_tag{};
    } else if constexpr (input_iterator<I>) {
      return input_iterator_tag{};
    }
  }

  template<class I>
  using bidirectional-at-most-t = decltype(bidirectional-at-most<I>()); // exposition
        only

}
```

§ 9.2 Transcoding views

```
namespace std::ranges {

  enum class utf_transcoding_error {
    truncated_utf8_sequence,
    unpaired_high_surrogate,
    unpaired_low_surrogate,
    unexpected_utf8_continuation_byte,
    overlong,
    encoded_surrogate,
    out_of_range,
    invalid_utf8_leading_byte
  };

  template<class V>
  concept from-utf-view = utf-range<V> && ranges::view<V>;

  template<bool OrError, code-unit ToType, from-utf-view V>
  class to-utf-view-impl {
  public:
    template<bool Const>
```

```
  class utf-iterator {
  private:
    using iter = ranges::iterator_t<maybe-const<Const, V>>;
    using sent = ranges::sentinel_t<maybe-const<Const, V>>;

    template<bool OrError2, code-unit ToType2,
             from-utf-view V2>
    friend class to-utf-view-impl; // exposition only

    using innermost-iter = unspecified; // exposition only

    using from-type = iter_value_t<iter>; // exposition only

  public:
     using value_type = conditional_t<OrError, expected<ToType, utf_transcoding_er-
       ror>, ToType>;
    using reference_type = value_type;
    using difference_type = ptrdiff_t;
    using iterator_concept = bidirectional-at-most-t<iter>;

    constexpr utf-iterator() requires default_initializable<V> = default;

  private:
     constexpr utf-iterator(to-utf-view-impl const* parent, iter begin) // exposi-
       tion only
         : backptr_(parent), base_(move(begin)) {
       if (base() != end())
         read();
    }

    constexpr utf-iterator(to-utf-view-impl const* parent) // exposition only
        : backptr_(parent), base_(end()) {
      if (base() != end())
        read();
    }

  public:
    constexpr utf-iterator() = default;
    constexpr utf-iterator(utf-iterator const&) requires copyable<iter> = default;

     constexpr utf-iterator& operator=(utf-iterator const&) requires copyable<iter>
       = default;

    constexpr utf-iterator(utf-iterator&&) = default;

    constexpr utf-iterator& operator=(utf-iterator&&) = default;

    constexpr iter& base() & { return base_; }

    constexpr iter const& base() const& { return base_; }

    constexpr iter base() && { return move(base_); }

      constexpr expected<void, utf_transcoding_error> success() const noexcept re-
       quires(OrError); // exposition only

    constexpr value_type operator*() const;

    constexpr void advance-one() // exposition only
      requires forward_iterator<iter>
    {
      if (buf_index_ + 1 < buf_last_) {
        ++buf_index_;
```

```
    } else if (buf_index_ + 1 == buf_last_) {
      advance(base(), to_increment_);
      to_increment_ = 0;
      if (base() != end()) {
        read();
      } else {
        buf_index_ = 0;
      }
    }
  }

  constexpr void advance-one() // exposition only
    requires (!forward_iterator<iter>)
  {
    if (buf_index_ + 1 == buf_last_ && base() != end()) {
      read();
    } else if (buf_index_ + 1 <= buf_last_) {
      ++buf_index_;
    }
  }

  constexpr utf-iterator& operator++() requires (OrError)
  {
    if (!success()) {
      assert(buf_index_ == 0);
      if constexpr (is_same_v<ToType, char8_t>) {
        advance-one();
        advance-one();
      }
    }
    advance-one();
    return *this;
  }

  constexpr utf-iterator& operator++() requires (!OrError)
  {
    advance-one();
    return *this;
  }

  constexpr auto operator++(int) {
    if constexpr (is_same_v<iterator_concept, input_iterator_tag>) {
      ++*this;
    } else {
      auto retval = *this;
      ++*this;
      return retval;
    }
  }

  constexpr utf-iterator& operator--() requires bidirectional_iterator<iter>
  {
    if (!buf_index_)
      read_reverse();
    else if (buf_index_)
      --buf_index_;
    return *this;
  }

  constexpr utf-iterator operator--(int) requires bidirectional_iterator<iter>
  {
    auto retval = *this;
```

```
      --*this;
      return retval;
    }

     friend constexpr bool operator==(utf-iterator const& lhs, utf-iterator const&
      rhs)
      requires forward_iterator<iter> || requires (iter i) { i != i; }
    {
      if constexpr (forward_iterator<iter>) {
        return lhs.base() == rhs.base() && lhs.buf_index_ == rhs.buf_index_;
      } else {
        if (lhs.base() != rhs.base())
          return false;

        if (lhs.buf_index_ == rhs.buf_index_ && lhs.buf_last_ == rhs.buf_last_) {
          return true;
        }

        return lhs.buf_index_ == lhs.buf_last_ && rhs.buf_index_ == rhs.buf_last_;
      }
    }

     friend constexpr bool operator==(utf-iterator const& lhs, sent rhs) requires
      copyable<iter>
    {
      if constexpr (forward_iterator<iter>) {
        return lhs.base() == rhs;
      } else {
        return lhs.base() == rhs && lhs.buf_index_ == lhs.buf_last_;
      }
    }

     friend constexpr bool operator==(utf-iterator const& lhs, sent rhs) requires
      (!copyable<iter>)
    {
      return lhs.base() == rhs && lhs.buf_index_ == lhs.buf_last_;
    }


    constexpr iter begin() const // exposition only
      requires bidirectional_iterator<iter>
    {
      return ranges::begin(backptr_->base_);
    }
    constexpr sent end() const { // exposition only
      return ranges::end(backptr_->base_);
    }

    constexpr void read(); // exposition only

    constexpr void read_reverse(); // exposition only
    array<value_type, 4 / sizeof(ToType)> buf_{}; // exposition only

    to-utf-view-impl const* backptr_;

    iter base_;

    uint8_t buf_index_ = 0; // exposition only
    uint8_t buf_last_ = 0; // exposition only
    uint8_t to_increment_ = 0; // exposition only

  };
```

```
private:
  V base_ = V(); // exposition only

  template<bool Const>
   static constexpr auto make_begin(to-utf-view-impl const* self, auto first) { //
      exposition only
    if constexpr (bidirectional_iterator<ranges::iterator_t<V>>) {
      return utf-iterator<Const>(self, first);
    } else {
      return utf-iterator<Const>(self, move(first));
    }
  }
  template<bool Const>
   static constexpr auto make_end(to-utf-view-impl const* self, auto last) { // ex-
      position only
    if constexpr (bidirectional_iterator<ranges::sentinel_t<V>>) {
      return utf-iterator<Const>(self);
    } else {
      return last;
    }
  }

public:
  constexpr to-utf-view-impl() requires default_initializable<V> = default;
  constexpr to-utf-view-impl(V base) : base_(move(base)) { }

  constexpr V base() const& requires copy_constructible<V>
  {
    return base_;
  }
  constexpr V base() && { return move(base_); }

  constexpr auto begin() requires (!copyable<ranges::iterator_t<V>>)
  {
    return make_begin<false>(this, ranges::begin(base_));
  }
  constexpr auto begin() const requires copyable<ranges::iterator_t<V>>
  {
    return make_begin<true>(this, ranges::begin(base_));
  }

  constexpr auto end() requires (!copyable<ranges::iterator_t<V>>)
  {
    return make_end<false>(this, ranges::end(base_));
  }
  constexpr auto end() const requires copyable<ranges::iterator_t<V>>
  {
    return make_end<true>(this, ranges::end(base_));
  }

  constexpr bool empty() const { return ranges::empty(base_); }
};

template<from-utf-view V>
class to_utf8_view : public ranges::view_interface<to_utf8_view<V>> {
private:
  using iterator = ranges::iterator_t<to-utf-view-impl<false, char8_t, V>>;
  using sentinel = ranges::sentinel_t<to-utf-view-impl<false, char8_t, V>>;

public:
  constexpr to_utf8_view() requires default_initializable<V> = default;
```

```
  constexpr to_utf8_view(V base) : impl_(move(base)) { }

  constexpr V base() const& requires copy_constructible<V>
  {
    return impl_.base();
  }
  constexpr V base() && { return move(impl_).base(); }

  constexpr auto begin() requires (!copyable<iterator>)
  {
    return impl_.begin();
  }
  constexpr auto begin() const requires copyable<iterator>
  {
    return impl_.begin();
  }

  constexpr auto end() requires (!copyable<iterator>)
  {
    return impl_.end();
  }
  constexpr auto end() const requires copyable<iterator>
  {
    return impl_.end();
  }

  constexpr bool empty() const { return impl_.empty(); }

private:
  to-utf-view-impl<false, char8_t, V> impl_;
};

template<class R>
to_utf8_view(R&&) -> to_utf8_view<views::all_t<R>>;

template<from-utf-view V>
   class   to_utf8_or_error_view   :   public   ranges::view_interface<to_utf8_or_er-
      ror_view<V>> {
private:
  using iterator = ranges::iterator_t<to-utf-view-impl<true, char8_t, V>>;
  using sentinel = ranges::sentinel_t<to-utf-view-impl<true, char8_t, V>>;

public:
  constexpr to_utf8_or_error_view() requires default_initializable<V> = default;
  constexpr to_utf8_or_error_view(V base) : impl_(move(base)) { }

  constexpr V base() const& requires copy_constructible<V>
  {
    return impl_.base();
  }
  constexpr V base() && { return move(impl_).base(); }

  constexpr auto begin() requires (!copyable<iterator>)
  {
    return impl_.begin();
  }
  constexpr auto begin() const requires copyable<iterator>
  {
    return impl_.begin();
  }

  constexpr auto end() requires (!copyable<iterator>)
```

```
  {
    return impl_.end();
  }
  constexpr auto end() const requires copyable<iterator>
  {
    return impl_.end();
  }

  constexpr bool empty() const { return impl_.empty(); }

private:
  to-utf-view-impl<true, char8_t, V> impl_;
};

template<class R>
to_utf8_or_error_view(R&&) -> to_utf8_or_error_view<views::all_t<R>>;

template<from-utf-view V>
class to_utf16_view : public ranges::view_interface<to_utf16_view<V>> {
private:
  using iterator = ranges::iterator_t<to-utf-view-impl<false, char16_t, V>>;
  using sentinel = ranges::sentinel_t<to-utf-view-impl<false, char16_t, V>>;

public:
  constexpr to_utf16_view() requires default_initializable<V> = default;
  constexpr to_utf16_view(V base) : impl_(move(base)) { }

  constexpr V base() const& requires copy_constructible<V>
  {
    return impl_.base();
  }
  constexpr V base() && { return move(impl_).base(); }

  constexpr auto begin() requires (!copyable<iterator>)
  {
    return impl_.begin();
  }
  constexpr auto begin() const requires copyable<iterator>
  {
    return impl_.begin();
  }

  constexpr auto end() requires (!copyable<iterator>)
  {
    return impl_.end();
  }
  constexpr auto end() const requires copyable<iterator>
  {
    return impl_.end();
  }

  constexpr bool empty() const { return impl_.empty(); }

private:
  to-utf-view-impl<false, char16_t, V> impl_;
};

template<class R>
to_utf16_view(R&&) -> to_utf16_view<views::all_t<R>>;

template<from-utf-view V>
```

```
   class  to_utf16_or_error_view  :  public  ranges::view_interface<to_utf16_or_er-
       ror_view<V>> {
private:
  using iterator = ranges::iterator_t<to-utf-view-impl<true, char16_t, V>>;
  using sentinel = ranges::sentinel_t<to-utf-view-impl<true, char16_t, V>>;

public:
  constexpr to_utf16_or_error_view() requires default_initializable<V> = default;
  constexpr to_utf16_or_error_view(V base) : impl_(move(base)) { }

  constexpr V base() const& requires copy_constructible<V>
  {
    return impl_.base();
  }
  constexpr V base() && { return move(impl_).base(); }

  constexpr auto begin() requires (!copyable<iterator>)
  {
    return impl_.begin();
  }
  constexpr auto begin() const requires copyable<iterator>
  {
    return impl_.begin();
  }

  constexpr auto end() requires (!copyable<iterator>)
  {
    return impl_.end();
  }
  constexpr auto end() const requires copyable<iterator>
  {
    return impl_.end();
  }

  constexpr bool empty() const { return impl_.empty(); }

private:
  to-utf-view-impl<true, char16_t, V> impl_;
};

template<class R>
to_utf16_or_error_view(R&&) -> to_utf16_or_error_view<views::all_t<R>>;

template<from-utf-view V>
class to_utf32_view : public ranges::view_interface<to_utf32_view<V>> {
private:
  using iterator = ranges::iterator_t<to-utf-view-impl<false, char32_t, V>>;
  using sentinel = ranges::sentinel_t<to-utf-view-impl<false, char32_t, V>>;

public:
  constexpr to_utf32_view() requires default_initializable<V> = default;
  constexpr to_utf32_view(V base) : impl_(move(base)) { }

  constexpr V base() const& requires copy_constructible<V>
  {
    return impl_.base();
  }
  constexpr V base() && { return move(impl_).base(); }

  constexpr auto begin() requires (!copyable<iterator>)
  {
    return impl_.begin();
```

```
    }
    constexpr auto begin() const requires copyable<iterator>
    {
      return impl_.begin();
    }

    constexpr auto end() requires (!copyable<iterator>)
    {
      return impl_.end();
    }
    constexpr auto end() const requires copyable<iterator>
    {
      return impl_.end();
    }

    constexpr bool empty() const { return impl_.empty(); }

  private:
    to-utf-view-impl<false, char32_t, V> impl_;
  };

  template<class R>
  to_utf32_view(R&&) -> to_utf32_view<views::all_t<R>>;

  template<from-utf-view V>
     class  to_utf32_or_error_view  :  public  ranges::view_interface<to_utf32_or_er-
         ror_view<V>> {
  private:
    using iterator = ranges::iterator_t<to-utf-view-impl<true, char32_t, V>>;
    using sentinel = ranges::sentinel_t<to-utf-view-impl<true, char32_t, V>>;

  public:
    constexpr to_utf32_or_error_view() requires default_initializable<V> = default;
    constexpr to_utf32_or_error_view(V base) : impl_(move(base)) { }

    constexpr V base() const& requires copy_constructible<V>
    {
      return impl_.base();
    }
    constexpr V base() && { return move(impl_).base(); }

    constexpr auto begin() { return impl_.begin(); }
    constexpr auto begin() const { return impl_.begin(); }

    constexpr auto end() { return impl_.end(); }
    constexpr auto end() const { return impl_.end(); }

    constexpr bool empty() const { return impl_.empty(); }

  private:
    to-utf-view-impl<true, char32_t, V> impl_;
  };

  template<class R>
  to_utf32_or_error_view(R&&) -> to_utf32_or_error_view<views::all_t<R>>;

namespace views {

  template<code-unit-to ToType>
  inline constexpr unspecified to_utf;

  template<code-unit-to ToType>
```

```
  inline constexpr unspecified to_utf_or_error;

  inline constexpr unspecified to_utf8;

  inline constexpr unspecified to_utf8_or_error;

  inline constexpr unspecified to_utf16;

  inline constexpr unspecified to_utf16_or_error;

  inline constexpr unspecified to_utf32;

  inline constexpr unspecified to_utf32_or_error;
}

}
```

`to_utf8_view` produces a view of the UTF-8 code units transcoded from the elements of a *`utf-range`*. `to_utf16_view` produces a view of the UTF-16 code units transcoded from the elements of a *`utf-range`*. `to_utf32_view` produces a view of the UTF-32 code units transcoded from the elements of a *`utf-range`*. Their `or_error` equivalents produce a view of `expected<charN_t, utf_transcoding_error>` where invalid input subsequences result in errors.

`to-utf-view-impl` is an exposition-only class that provides implementation details common to the six aforementioned transcoding views.

The iterator type of *`to-utf-view-impl`* is *`utf-iterator`*. *`utf-iterator`* is an iterator that transcodes from UTF-N to UTF-M, where N and M are each one of 8, 16, or 32. N may equal M.

`utf-iterator` uses a mapping between character types and UTF encodings, which is that that `char8_t` corresponds to UTF-8, `char16_t` corresponds to UTF-16, and `char32_t` corresponds to UTF-32.

`utf-iterator` does its work by adapting an underlying range of code units. We use the term "input subsequence" to refer to a potentially ill-formed code unit subsequence which is to be transcoded into a code point `c`. Each input subsequence is decoded from the UTF encoding corresponding to *`from-type`*. If the underlying range contains ill-formed UTF, the code units are divided into input subsequences according to Substitution of Maximal Subparts, and each ill-formed input subsequence is transcoded into a `U+FFFD`. `c` is then encoded to `ToType`'s corresponding encoding, into an internal code unit buffer.

`utf-iterator` maintains certain invariants; the invariants differ based on whether *`utf-iterator`* is an input iterator.

For input iterators the invariant is: if `*this` is at the end of the range being adapted, then `base()` == *`end()`*; otherwise, the position of `base()` is always at the end of the input subsequence corresponding to the current code point `c`, and `buf_` contains the code units that comprise `c`, in the UTF encoding corresponding to `ToType`.

For forward and bidirectional iterators, the invariant is: if `*this` is at the end of the range being adapted, then `base()` == *`end()`*; otherwise, the position of `base()` is always at the beginning of the input subsequence corresponding to the current code point `c` within the underlying range, and `buf_` contains the code units in `ToFormat` that comprise `c`.

The exposition-only member function `read` decodes the input subsequence starting at position `base()` into a code point `c`, using the UTF encoding corresponding to *`from-type`*, and setting `c` to U+FFFD if the input subsequence is ill-formed. If `c` is set to U+FFFD as the result of an ill-formed input subse-

quence, it sets the error as described below. It sets `to_increment_` to the number of code units read while decoding `c`; encodes `c` into `buf_` in the UTF encoding corresponding to `ToType`; sets `buf_index_` to `0`; and sets `buf_last_` to the number of code units encoded into `buf_`. If `forward_iterator<I>` is `true`, `base()` is set to the position it had before `read` was called.

The exposition-only member function `read_reverse` decodes the input subsequence ending at position `base()` into a code point `c`, using the UTF encoding corresponding to *`from-type`*, and setting `c` to U+FFFD if the input subsequence is ill-formed. If `c` is set to U+FFFD as the result of an ill-formed input subsequence, it sets the error as described below. It sets `to_increment_` to the number of code units read while decoding `c`; encodes `c` into `buf_` in the UTF encoding corresponding to `ToType`; sets `buf_last_` to the number of code units encoded into `buf_`; and sets `buf_index_` to `buf_last_ - 1`, or to `0` if this is an `or_error` view and we read an invalid subsequence.

If the view is an `or_error` view that encountered an invalid subsequence, that subsequence becomes a single `value_type` set to a `utf_transcoding_error` enumerator in the output range. The value of the enumerator corresponds to the underlying range's input subsequences as follows. (All ranges of numerical values of code units below are inclusive.)

- If the encoding corresponding to *`from-type`* is UTF-8:
  - If the current input subsequence is a code unit between 0x80 and 0xBF, the error is `unexpected_utf8_continuation_byte`.
  - If the current input subsequence is a code unit between 0xC0 and 0xC2, or between 0xF5 and 0xFF, the error is `invalid_utf8_leading_byte`.
  - If the current input subsequence is 0xE0, and the subsequent input subsequence is between 0x80 and 0x9F; or if the current input subsequence is 0xF0, and the subsequent input subsequence is between 0x80 and 0x8F; then the error is `overlong`.
  - If the current input subsequence is 0xED, and the subsequent input subsequence is between 0xA0 and 0xBF, then the error is `encoded_surrogate`.
  - If the the current input subsequence is 0xF4, and the subsequent input subsequence is between 0x90 and 0xBF, then the error is `out_of_range`
  - Otherwise, if the current input subsequence is invalid UTF-8, begins with a code unit between 0xC2 and 0xF4, and there exists some hypothetical sequence of code units which would make the current input subsequence well-formed if concatenated to the end of it, then the error is `truncated_utf8_sequence`.
- If the encoding corresponding to *`from-type`* is UTF-16:
  - If the current input subsequence is between 0xD800 and 0xDBFF, the error is `unpaired_high_surrogate`.
  - If the current input subsequence is between 0xDC00 and 0xDFFF, the error is `unpaired_low_surrogate`.
- If the encoding corresponding to *`from-type`* is UTF-32:
  - If the current input subsequence is between 0xD800 and 0xDFFF, the error is `encoded_surrogate`.
  - If the current input subsequence is between 0x110000 and 0xFFFFFFFF, the error is `out_of_range`.

The exposition-only member function *success* returns `false` if the current input subsequence is invalid, `true` otherwise.

If *utf-iterator* is a `bidirectional_iterator`, it is defined to be at the beginning of its underlying range if `buf_index_` is zero and `base() == begin()`. If it is a `forward_iterator`, it is defined to be at the end of its underlying range if `buf_index_ + 1 == buf_last_` and `base() == end()`. Otherwise, it is defined to be at the end of its underlying range if `buf_index_ == buf_last_` and `base() == end()`.

If `operator*` or `operator++` are invoked while *utf-iterator* is at the end of its underlying range, the behavior is undefined; if `operator--` is invoked while *utf-iterator* is at the beginning of its underlying range, the behavior is undefined.

The names `to_utf8`, `to_utf8_or_error`, `to_utf16`, `to_utf16_or_error`, `to_utf32`, and `to_utf32_or_error` denote range adaptor objects ([range.adaptor.object]). `to_utf` and `to_utf_or_error` denote range adaptor object templates. `to_utfN` produces `to_utfN_views`, and `to_utfN_or_error` produces `to_utfN_or_error_views`. `to_utf<ToType>` is equivalent to `to_utf8` if `ToType` is `char8_t`, `to_utf16` if `ToType` is `char16_t`, and `to_utf32` if `ToType` is `char32_t`, and similarly for `to_utf_or_error`. Let `to_utfN` denote any of the aforementioned range adaptor objects, let `Char` be its corresponding character type, and let `V` denote the `to_utfN_view` or 2`to_utfN_or_error_view` associated with that object. Let `E` be an expression and let `T` be `remove_cvref_t<decltype((E))>`. If `decltype((E))` does not model *utf-range*, `to_utfN(E)` is ill-formed. The expression `to_utfN(E)` is expression-equivalent to:

— If `E` is a specialization of `empty_view` ([range.empty.view]), then `empty_view<Char>{}`.

— Otherwise, if `T` is an array type of known bound, then:

  — If the array extent is nonzero and the last element of the array is zero, then `V(std::ranges::subrange(std::ranges::begin(E), --std::ranges::end(E)))`

  — Otherwise, `V(std::ranges::subrange(std::ranges::begin(E), std::ranges::end(E)))`

— Otherwise, `V(std::views::all(E))`

The implementation of the `empty()` member function provided by the transcoding views is more efficient than the one provided by `view_interface`, since `view_interface`'s implementation will construct `utf_view::begin()` and `utf_view::end()` and compare them, whereas we can simply use the underlying range's `empty()`, since a transcoding view is empty if and only if its underlying range is empty.

§ 9.3 Code unit adaptors

```
namespace std::ranges::views {

  template<class T>
  struct implicit-cast-to {
    constexpr T operator()(auto x) const noexcept { return x; }
  };

  inline constexpr unspecified as_char8_t;

  inline constexpr unspecified as_char16_t;

  inline constexpr unspecified as_char32_t;
}
```

The names `as_char8_t`, `as_char16_t`, and `as_char32_t` denote range adaptor objects ([range.adaptor.object]). Let `as_charN_t` denote any one of `as_char8_t`, `as_char16_t`, and `as_char32_t`. Let `Char` be the corresponding character type for `as_charN_t`, let `E` be an expression and let `T` be `remove_cvref_t<decltype((E))>`. If `ranges::range_reference_t<T>` does not model `convertible_to<Char>`, `as_charN_t(E)` is ill-formed. The expression `as_charN_t(E)` is expression-equivalent to:

— If `T` is a specialization of `empty_view` ([range.empty.view]), then `empty_view<Char>{}`.

— Otherwise, if `T` is an array type of known bound, then:

  — If the array extent is nonzero and the last element of the array is zero, then `ranges::transform_view(std::ranges::subrange(std::ranges::begin(E), std::ranges::end(E)), `*`implicit-cast-to`*`<Char>{})`

  — Otherwise, `ranges::transform_view(std::ranges::subrange(std::ranges::begin(E), std::ranges::end(E)), `*`implicit-cast-to`*`<Char>{})`

— Otherwise, `ranges::transform_view(std::views::all(E), `*`implicit-cast-to`*`<Char>{})`

[Example 1:

```
std::vector<int> path_as_ints = {U'C', U':', U'\x00010000'};
std::filesystem::path path = path_as_ints | as_char32_t |
        std::ranges::to<std::u32string>();
auto const& native_path = path.native();
if (native_path != std::wstring{L'C', L':', L'\xD800', L'\xDC00'}) {
  return false;
}
```

— end example]

§ 9.4 Feature test macro

Add the feature test macro `__cpp_lib_unicode_transcoding`.

§ 10 Changelog

§ 10.1 Changes since R7

— Add playing card example.

— Remove `iterator_interface` from *`utf-iterator`*.

— Replace code unit views with range adaptor closure objects that are expression-equivalent to P3117 `transform_view`.

— Move `null_sentinel` and `null_term` into P3705.

— Remove `std::uc` namespace and replace it with `std::ranges` and `std::ranges::views`.

— Remove library EB.

— Change iterator to use a back-pointer to its parent view, removing borrowedness and quadratic stack size growth.

— Remove support for `char` and `wchar_t`.

— Rewrite most of the verbiage and add new diagrams.

§ 10.2 Changes since R6

— Fix a bug in `null_sentinel_t` causing it not to satisfy `sentinel_for` by changing its `operator==` to return `bool`.

— Fix a bug in `null_sentinel_t` where it did not support non-copyable input iterators by having operator== take input iterators by reference.

— Rename `as_utfN` to `to_utfN` to emphasize that a conversion is taking place and to contrast with the code unit views, which remain named `as_charN_t`.

— Refactor `utf_view` into an exposition-only *`utf-view-impl`* class used as an implementation detail of separate `to_utf8_view`, `to_utf16_view`, and `to_utf32_view` classes, addressing broken deduction guides in the previous revision.

— Remove `project_view` and copy most of its implementation into separate `char8_view`, `char16_view`, and `char32_view` classes, addressing broken deduction guides in the previous revision.

— Change `utf_iterator` to an exposition-only member class of *`utf-view-impl`*.

— Eliminate iterator unpacking mechanism and replace it with an alternative solution to the problem of transcoding ranges wrapping other transcoding ranges. This simplifies the API at the expense of removing the transcoding iterator's `begin()` and `end()` member functions and losing the ability to implement unpacking for user-defined UTF iterators.

— Remove `std::uc::format`.

— Make all concepts exposition-only.

— Remove `utf_transcoding_error_handler` mechanism.

— Introduce new error handling mechanism based on a new `utf_transcoding_error` enumeration which is returned by an `success()` member function of the transcoding view's iterator.

— Remove ability to pass pointers to range adaptor closure objects, which violated the restriction that only ranges may be passed to range adaptor closure objects.

— Remove `std::format` and `std::ostream` functionality. It doesn't make sense for this mechanism to be the only way we have to format/output `char8_t`; we can revisit this functionality when we have already figured out how to support e.g. `std::u8string`.

— Replace code examples with new ones reflecting API changes.

— Provide a reference implementation.

§ 10.3 Changes since R5

— Simplify the complicated constraint on the comparison operator for `null_sentinel_t`.

— Introduce `ranges::project_view`, and implement `charN_views` in terms of that.

— Convert the `utfN_views` to aliases, rather than individual classes.

§ 10.4 Changes since R4

— Replace `unpacking_owning_view` with `unpacking_view`, and use it to do unpacking, rather than sometimes doing the unpacking in the adaptor.

— Ensure `const` and non-`const` overloads for `begin` and `end` in all views.

— Move `null_sentinel_t` to `std`, remove its `base` member function, and make it useful for more than just pointers, based on SG-9 guidance.

§ 10.5 Changes since R3

— Changed the definition of the `code_unit` concept, and added `as_charN_t` adaptors.

— Removed the utility functions and Unicode-related constants, except `replacement_character`.

— Changed the constraint on `utf_iterator` slightly.

— Change `null_sentinel_t` back to being Unicode-specific.

§ 10.6 Changes since R2

— Add `noexcept` where appropriate.

— Remove non-essential constants and utility functions, and elaborate on the usage of the ones that remain.

— Note differences from similar elements proposed in [P1629R1].

— Extend the examples slightly.

— Correct an error in the description of the view adaptors' semantics, and provide several examples of their use.

§ 10.7 Changes since R1

— Reintroduce the transcoding-from-a-buffer example.

— Generalize `null_sentinel_t` to a non-Unicode-specific facility.

— In utility functions that search for ill-formed encoding, take a range argument instead of a pair of iterator arguments.

— Replace `utf{8,16,32}_view` with a single `utf_view`.

§ 10.8 Changes since R0

— When naming code points in interfaces, use `char32_t`.

— When naming code units in interfaces, use `charN_t`.

— Remove each eager algorithm, leaving in its corresponding view.

— Remove all the output iterators.

— Change template parameters to `utfN_view` to the types of the from-range, instead of the types of the transcoding iterators used to implement the view.

— Remove all make-functions.

— Replace the misbegotten `as_utfN()` functions with the `as_utfN` view adaptors that should have been there all along.

— Add missing `utf_transcoding_error_handler` concept.

— Turn `unpack_iterator_and_sentinel` into a CPO.

— Lower the UTF iterator concepts from bidirectional to input.

§ 11 Relevant Polls/Minutes

§ 11.1 Unofficial SG9 review of P2728R7 during Wrocław 2024

SG9 members provided unofficial guidance that the `.success()` member function on the *`utf-iterator`* wasn't workable and encouraged providing views with `std::expected` as a value type.

§ 11.2 SG16 review of P2728R6 on 2023-09-13 (Telecon)

— Minutes

No polls were taken during this review.

§ 11.3 SG16 review of P2728R6 on 2023-08-23 (Telecon)

— Minutes

No polls were taken during this review.

§ 11.4 SG9 review of D2728R4 on 2023-06-12 during Varna 2023

— Minutes

POLL: utf_iterator should be a separate type and not nested within utf_view

SF	F	N	A	SA
1	2	1	0	1

Attendance: 8 (3 abstentions)

# of Authors: 1

Author Position: F

Outcome: Weak consensus in favor

SA: Having a separate type complexifies the API

§ 11.5 SG16 review of P2728R0 on 2023-04-12 (Telecon)

— Minutes

POLL: SG16 would like to see a version of P2728 without eager algorithms.

SF	F	N	A	SA
4	2	0	1	0

Attendance: 10 (3 abstentions)

Outcome: Consensus in favor

POLL: UTF transcoding interfaces provided by the C++ standard library should operate on charN_t types, with support for other types provided by adapters, possibly with a special case for char and wchar_t when their associated literal encodings are UTF.

SF	F	N	A	SA
5	1	0	0	1

Attendance: 9 (2 abstentions)

Outcome: Strong consensus in favor

Author's note: More commentary on this poll is provided in the section "Discussion of whether transcoding views should accept ranges of `char` and `wchar_t`". But note here that the authors doubt the viability of "a special case for char and wchar_t when their associated literal encodings are UTF", since making the evaluation of a concept change based on the literal encoding seems like a flaky move; the literal encoding can change TU to TU.

§ 11.6 SG16 review of P2728R0 on 2023-03-22 (Telecon)

— Minutes

No polls were taken during this review.

POLL: `char32_t` should be used as the Unicode code point type within the C++ standard library implementations of Unicode algorithms.

SF	F	N	A	SA
6	0	1	0	0

Attendance: 9 (2 abstentions)

Outcome: Strong consensus in favor

§ 12 Special Thanks

Zach Laine, for writing revisions one through six of the paper and implementing Boost.Text.

Jonathan Wakely, for implementing P2728R6, and design guidance.

Robert Leahy and Gašper Ažman, for design guidance.

§ 13 References

[CVE-2007-3917] NVD - CVE-2007-3917.
https://nvd.nist.gov/vuln/detail/CVE-2007-3917

[Definitions] The Unicode Standard, Version 17.0, §3.4 Characters and Encoding.
https://www.unicode.org/versions/Unicode17.0.0/core-spec/chapter-3/#G2212

[N2902] JeanHeyd Meneide. Restartable and Non-Restartable Functions for Efficient Character Conversions, revision 6.
https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2902.htm

[Null-terminated multibyte strings] Null-terminated multibyte strings.
https://en.cppreference.com/w/c/string/multibyte.html
[P1629R1] JeanHeyd Meneide. 2020-03-02. Transcoding the world - Standard Text Encoding.
https://wg21.link/p1629r1
[P1814R0] Mike Spertus. 2019-07-28. Wording for Class Template Argument Deduction for Alias Templates.
https://wg21.link/p1814r0
[P2871R3] Alisdair Meredith. 2023-12-18. Remove Deprecated Unicode Conversion Facets From C++26.
https://wg21.link/p2871r3
[P2873R2] Alisdair Meredith, Tom Honermann. 2024-07-06. Remove Deprecated locale category facets for Unicode from C++26.
https://wg21.link/p2873r2
[P3117R1] Zach Laine, Barry Revzin, Jonathan Müller. 2024-12-15. Extending Conditionally Borrowed.
https://wg21.link/p3117r1
[Substitution] The Unicode Standard, Version 17.0, §3.9.6 Substitution of Maximal Subparts.
https://www.unicode.org/versions/Unicode17.0.0/core-spec/chapter-3/#G66453
[SubstitutionExamples] The Unicode Standard, Version 17.0, §3.9.6 Substitution of Maximal Subparts.
https://www.unicode.org/versions/Unicode17.0.0/core-spec/chapter-3/#G67519
[wesnoth] The Battle for Wesnoth, “fixed a crash if the client recieves invalid utf-8.”
https://github.com/wesnoth/wesnoth/commit/c5bc4e2a915ddf53b63f292f587526aaa39a96aa